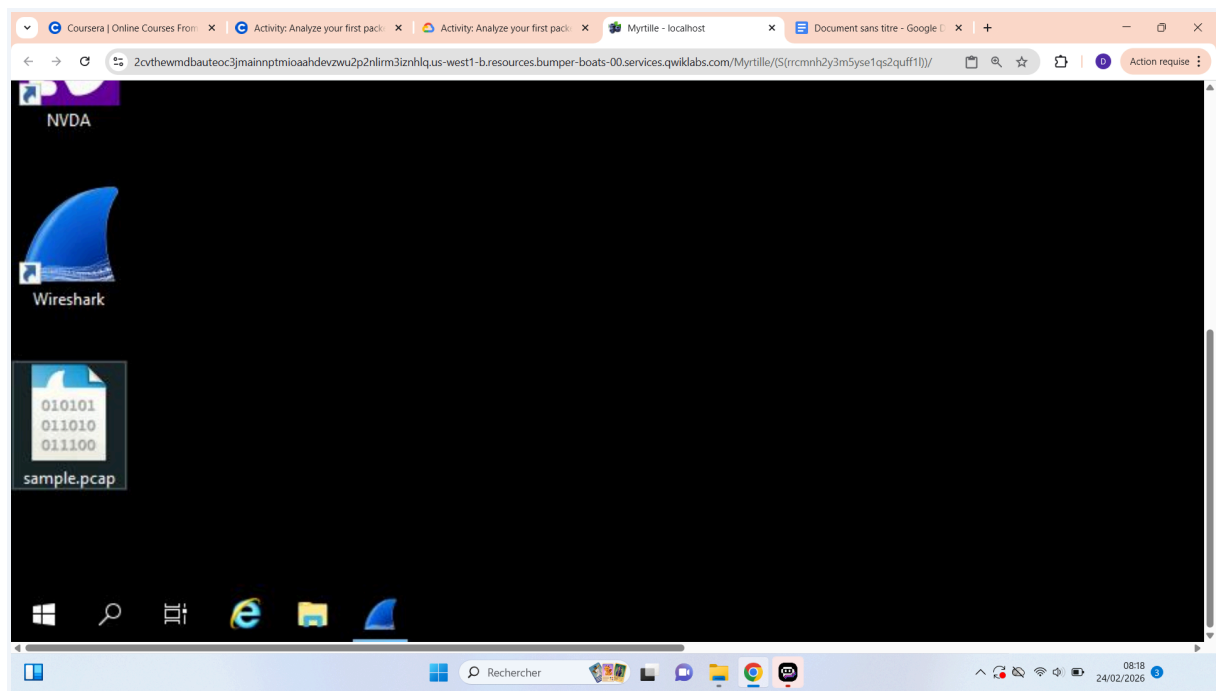


Packet analysing with Wireshark

Analyzing packets can help security teams interpret and understand network communications.

In this activity, I'll use Wireshark to understand the network traffic data by applying filters to sort through packet information efficiently.

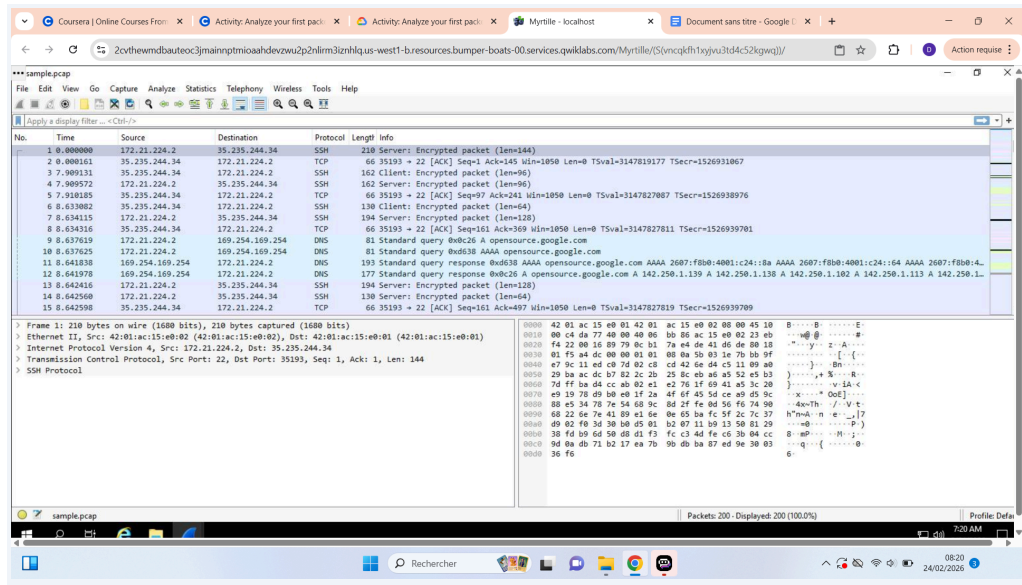
My resource for this activity will be a capture file named sample.pcap .



Apply a basic Wireshark filter and inspect a packet

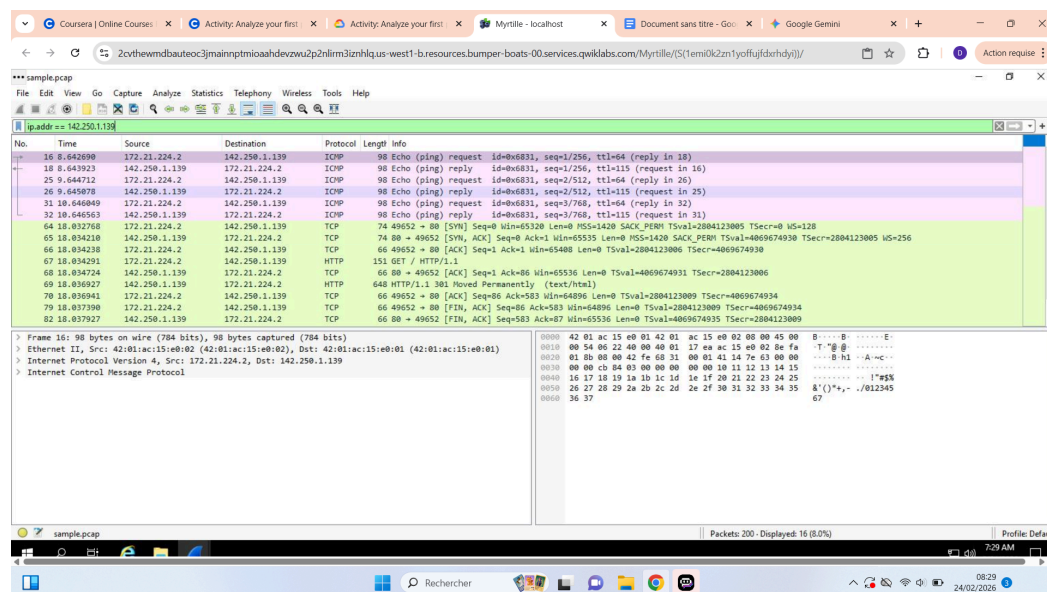
Here is the unfiltered window :

We have a list of lines where each line is a packet with different layers :number, time, source, destination, protocol, length and info.



ip.addr

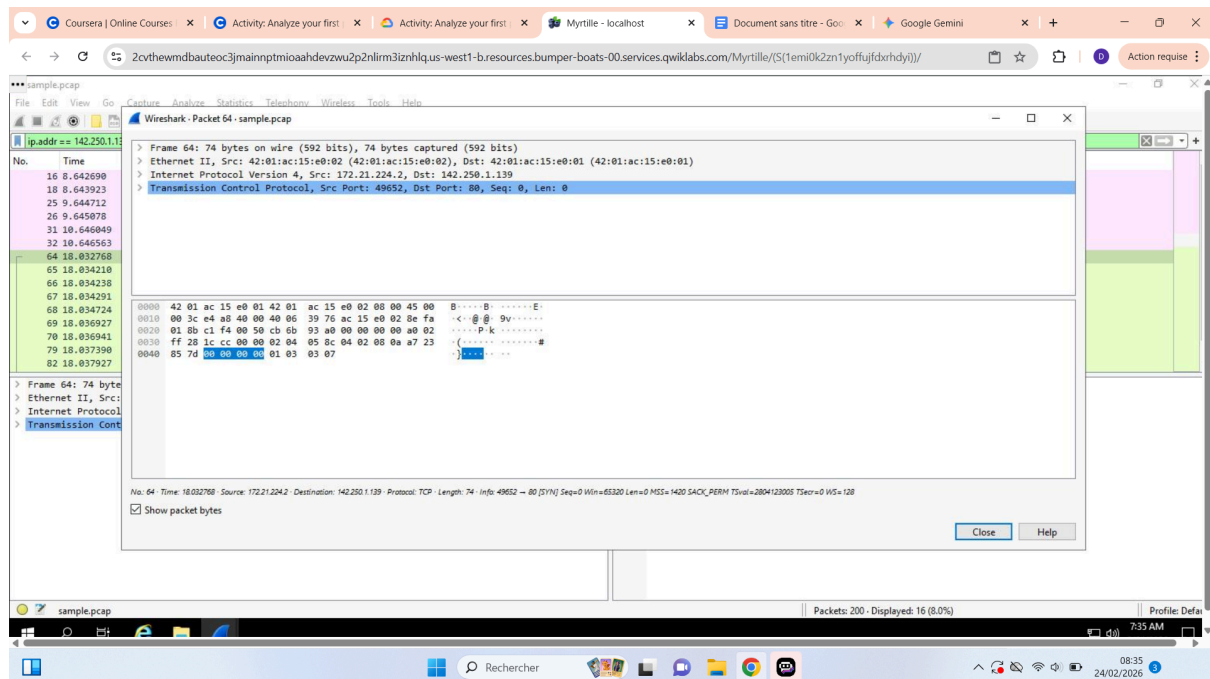
After applying an ip search filter such as **ip.addr == 142.250.1.139**, we get any traffic going in or out from this ip address .



Here , we could see ICMP packets highlighted in pink and TCP or HTTP highlighted in green.

Deep Packet Inspection

After double clicking the first packet that lists **TCP** as the protocol, this windows open showing more packet details.



The upper section of this window contains subtrees where Wireshark will provide an analysis of the various parts of the network packet.

There are **four different subtrees** displayed on the screen:

Frame :

This provides details about the overall network packet, or frame, including the frame length and the arrival time of the packet. At this level, we are viewing information about the entire packet of data.

Ethernet:

This item contains details about the packet at the Ethernet level, including the source and destination MAC addresses and the type of internal protocol that the Ethernet packet contains.

IP version:

This provides packet data about the Internet Protocol (IP) data contained in the Ethernet packet. It contains information such as the source and destination IP addresses and the Internal Protocol (for example, TCP or UDP), which is carried inside the IP packet.

And the type of protocol used , TCP for this packet :

This provides detailed information about the TCP packet, including the source and destination TCP ports, the TCP sequence numbers, and the TCP flags.

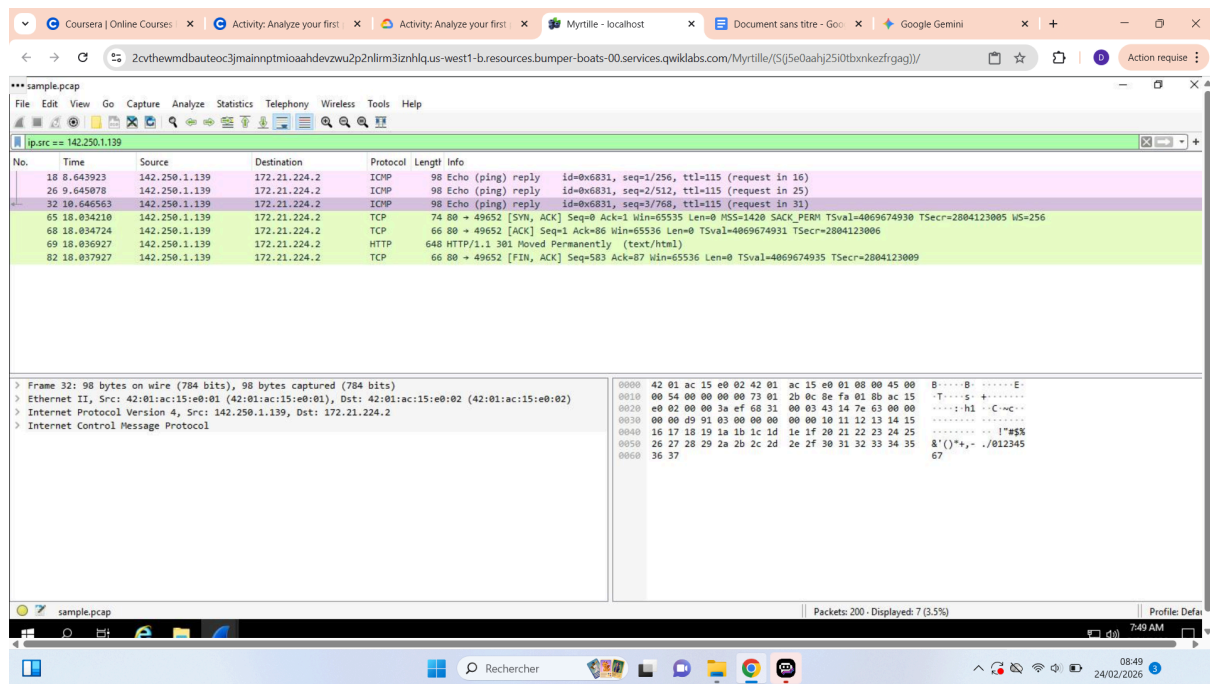
The source port and destination port listed here match the source and destination ports in the info column of the summary display for this packet in the list of all of the packets in the main Wireshark window.

Hexadecimal and ASCII text

The lower section of the window contains the raw packet data displayed in hexadecimal and ASCII text. There is also placeholder text for fields where the character data does not apply, as indicated by the dot (“.”).

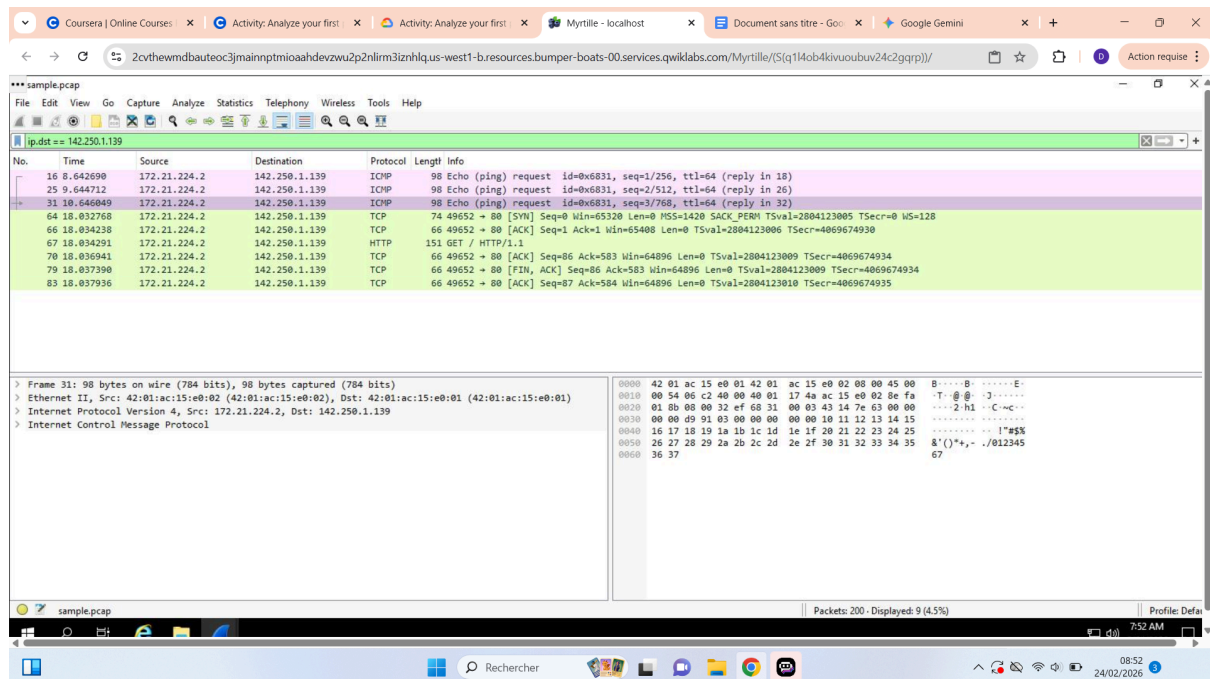
ip.src == 142.250.1.139

A filtered list is returned with fewer entries than before. It contains only packets that came from **142.250.1.139**. It's a more filtered search than ip.addr since it only list packet sent from this ip.



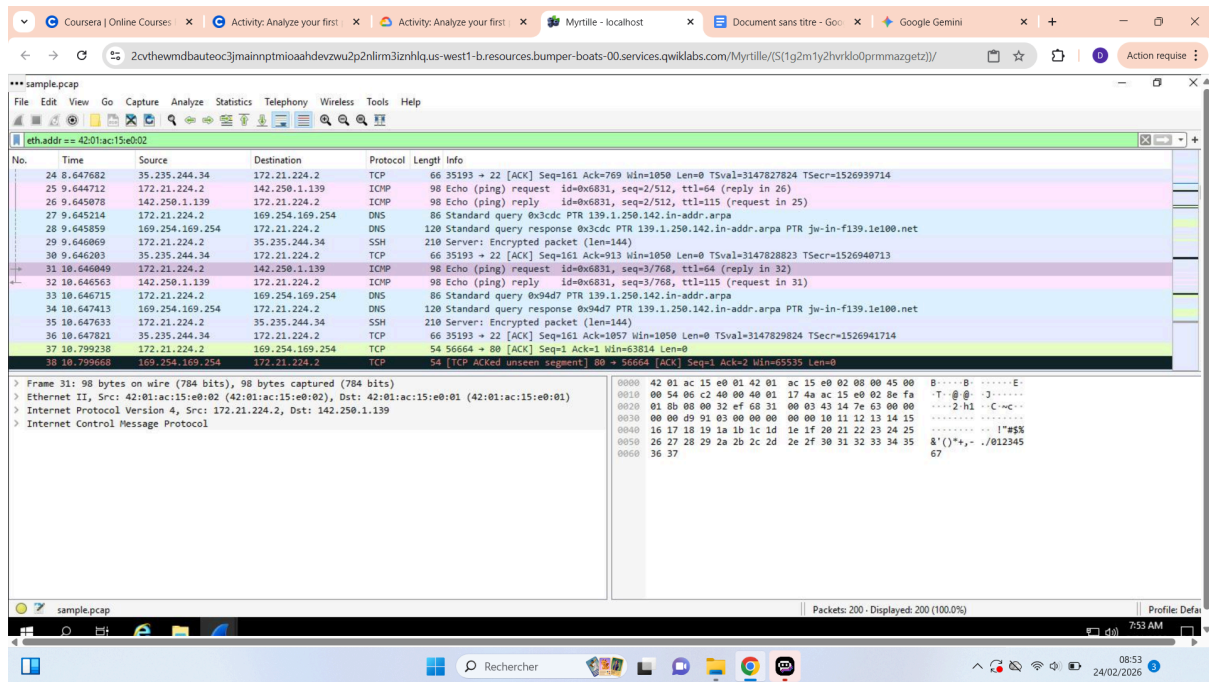
ip.dst == 142.250.1.139

A filtered list is returned that contains only packets that were sent to **142.250.1.139**.



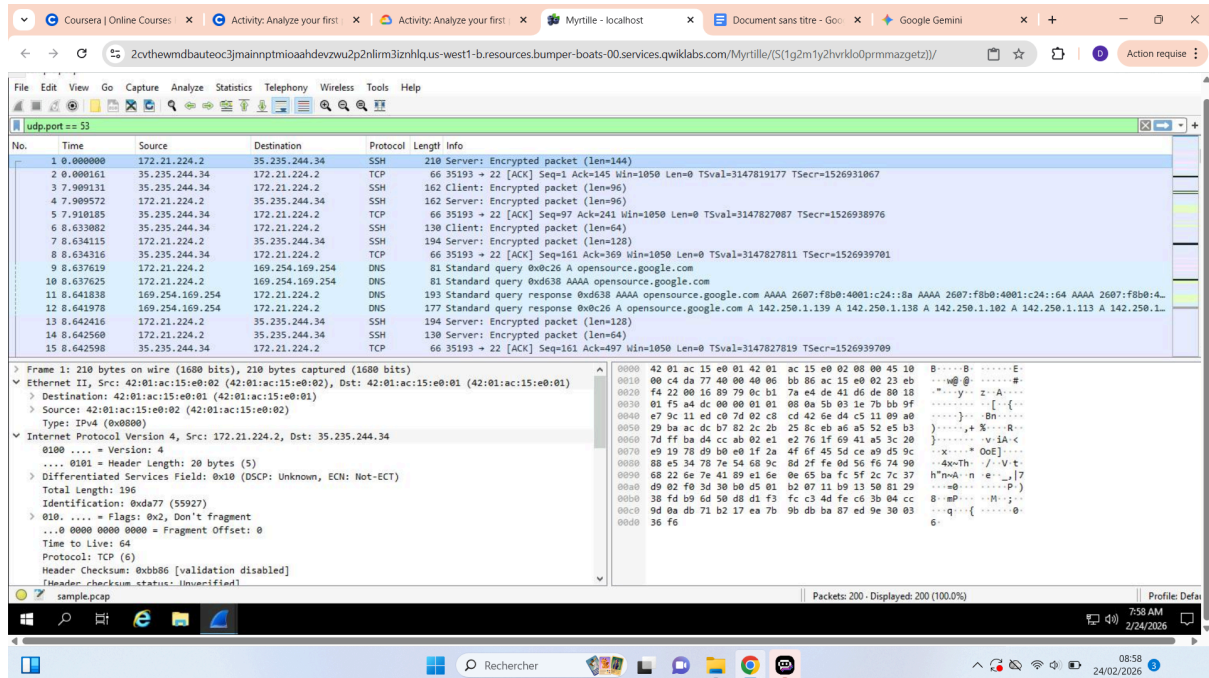
eth.addr == 42:01:ac:15:e0:02

Similar to ip.addr , eth.addr list all packet traffic for the mac address for any protocol



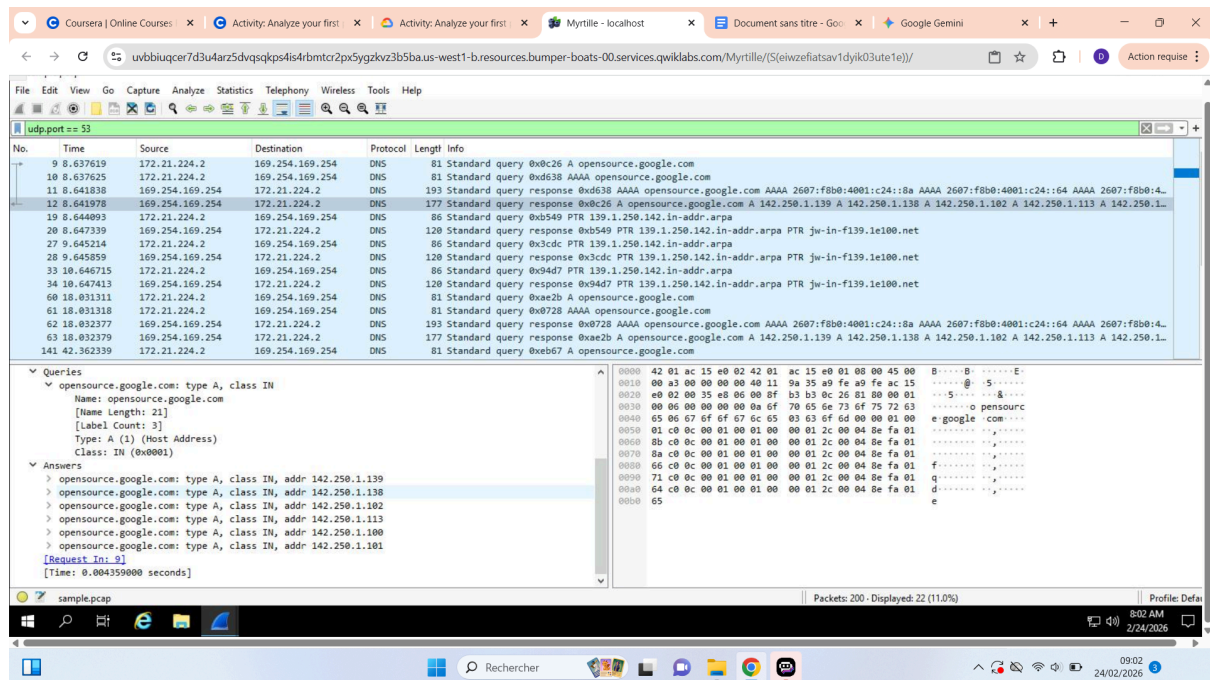
udp.port == 53

The following filter to select UDP port **53** traffic. DNS traffic uses UDP port **53**, so this will list traffic related to DNS queries and responses only.



DNS Query

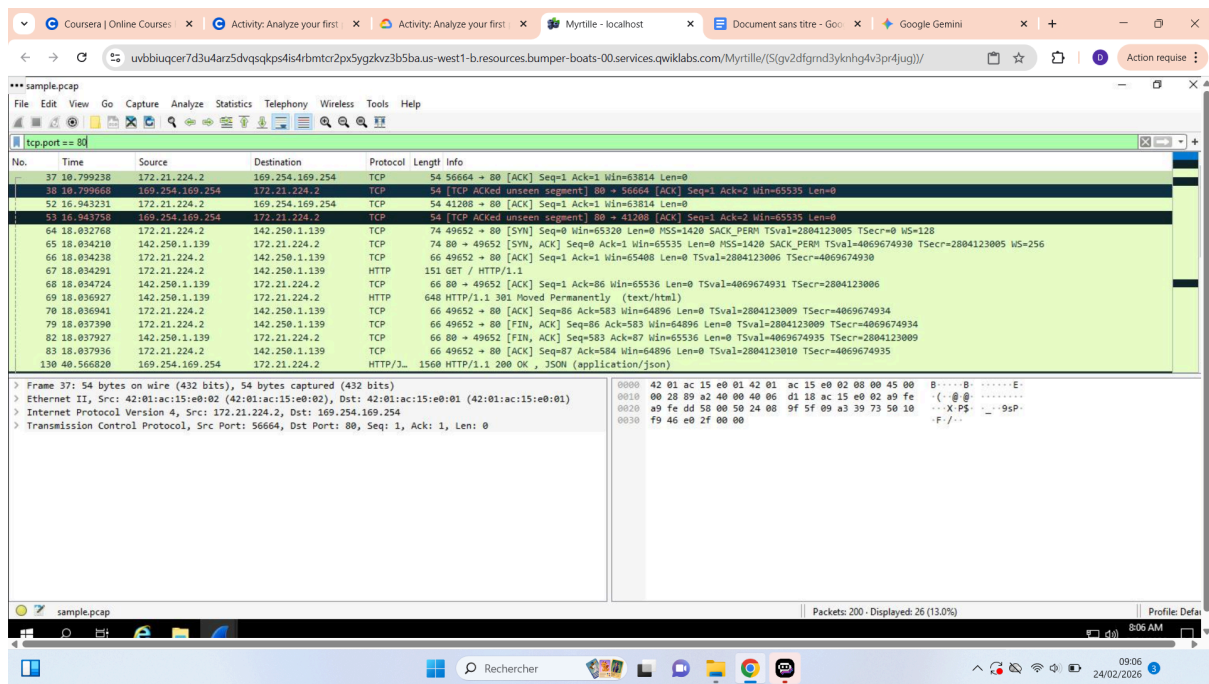
By looking at the DNS query subtree, and expanding answers, we could see the actual IP address in the expanded Answers section for the DNS query for opensource.google.com



Use filters to explore TCP packets

tcp.port == 80

TCP port **80** is the default port that is associated with web traffic



tcp contains "curl"

This filters to packets containing web requests made with the curl command in this sample packet capture file.

